Installation & Inventory

Kubernetes Production · Lesson 3

Built from the official documentation

`kubespray.io` · `kubernetes-sigs/kubespray`

Learning Objectives

By the end of this lesson you will be able to:

- **Get the Kubespray code** and pin it to a release tag
- Install Ansible deps into a **Python virtualenv** with `requirements.txt`
- Create your own inventory from `inventory/sample`
- Build `hosts.yaml` with the **inventory builder** and read its groups
- **Verify connectivity** before deploying, and know the collection method

> Everything happens on the **control node** — no agent runs on the targets.

3.1 Getting the Code

Clone + Check Out a Release Tag

On the control node, work inside a fresh Python virtualenv and **pin a tested release** — never deploy production off a moving `master`.

```
python3 -m venv venv
source venv/bin/activate

git clone https://github.com/kubernetes-sigs/kubespray.git
cd kubespray
git checkout release-2.17        # pin to a release tag/branch
```

- Ansible is a **Python application** — the venv isolates its deps.
- Kubespray ships **different `requirements.txt` per Ansible version**.

> Your Python version limits the usable Ansible/Kubespray version (recent releases: Python 3.11-3.13, Ansible >=2.18,<2.19).

Install the Dependencies

Install into the **active** virtualenv:

```
pip install -r requirements.txt
```

- Pulls the exact Ansible version that release of Kubespray was tested with.
- Keeps system Python untouched — no global package conflicts.

> If pip says it *cannot find* `ansible==X`, your Python is too old for this Kubespray tag — use a newer Python or an older release.

3.2 Creating Your Inventory

Copy the Sample Inventory

Copy `inventory/sample` to your own cluster dir (recursive, preserve attrs), then edit **only your copy** so upstream updates never clobber it.

```
cp -rfp inventory/sample inventory/mycluster
```

- Creates `inventory/mycluster/` with the inventory file **plus** the `group_vars/` tree (`group_vars/all/all.yml`, `group_vars/k8s_cluster/k8s-cluster.yml`, …).
- `group_vars/` is where Lesson 3 configures runtime / CNI / add-ons.

> Inventory may be **YAML, JSON, or INI** — pick whichever you prefer.

Defining Hosts in the Inventory

Edit the copied inventory and assign each node to its group(s):

```
<your-editor> inventory/mycluster/inventory.ini
```

```
[kube_control_plane]
node1
node2

[etcd]
node1
node2
node3

[kube_node]
node3
node4
```

> The old `contrib/inventory_builder/inventory.py` helper was **removed** upstream — define the inventory **by hand** (INI, YAML or JSON) or with your own tooling.

3.3 Inventory Structure

The Inventory Groups

Group	Role
kube_node	nodes where the pods run (workers)
kube_control_plane	apiserver, scheduler, controller-manager
etcd	etcd members — **at least 3** for failover
calico_rr	advanced Calico route-reflector cases
bastion	jump host when nodes have private IPs only

> **k8s_cluster** is **dynamically defined** = `kube_node` ∪ `kube_control_plane` ∪ `calico_rr` — used for cluster-wide `group_vars`.

Membership Rules

- Server in **both** `kube_control_plane` + `kube_node` → schedulable control plane (control plane *and* worker).
- Server **only** in `kube_control_plane` → standalone, **unschedulable** control plane.
- Server in **both** `kube_node` + `etcd` → etcd schedulable for workloads; keep groups disjoint for **standalone etcd**.

> Do **not** hand-edit the children of `k8s_cluster` — it is derived.

`hosts.yaml` — Per-Host Variables

```yaml
all:
  hosts:
    node1:
      ansible_host: 95.54.0.12   # IP Ansible SSHes to
      ip: 10.3.0.1               # IP k8s services bind to
      access_ip: 10.3.0.1        # IP other nodes use to reach it
  children:
    kube_control_plane:
      hosts: { node1: }
    etcd:
      hosts: { node1: }
    kube_node:
      hosts: { node2: }
    k8s_cluster:
      children: { kube_control_plane:, kube_node: }
```

> `ansible_host` = SSH target · `ip` = service bind · `access_ip` = node reach.

3.4 Verifying Connectivity

Ping Every Node First

Confirm Ansible can SSH **and** escalate to root on every host before the long deploy:

```
ansible -i inventory/mycluster/hosts.yaml all -m ping -b
```

- `all` = every host · `-m ping` = connectivity module · `-b` = become root.
- Connection vars: `ansible_user`, `ansible_become`, `ansible_ssh_private_key_file` — or pass `-u <user>` / `--private-key=...`.

> Privilege escalation (`-b` / `--become`) is required here **and** for the real deploy.

Deploying with `cluster.yml`

Same inventory drives the deploy (full coverage in Lesson 6) — `--become` is **mandatory**.

```
# YAML inventory from the builder
ansible-playbook -i inventory/mycluster/hosts.yaml \
  --become --become-user=root cluster.yml

# INI inventory with explicit user + key
ansible-playbook -i inventory/mycluster/inventory.ini \
  -u $USERNAME -b -v --private-key=~/.ssh/id_rsa cluster.yml
```

> Without `--become` the playbook fails: it writes to `/etc`, installs packages, and manages systemd daemons.

3.5 Kubespray as a Collection

Consume Kubespray as a Collection

Reference Kubespray from your **own** Ansible repo instead of a clone.

```
# requirements.yml
collections:
  - name: https://github.com/kubernetes-sigs/kubespray
    type: git
    version: master   # use the tag/branch you need
```

```
ansible-galaxy install -r requirements.yml
```

```
# your playbook.yml
- name: Install Kubernetes
  ansible.builtin.import_playbook: kubernetes_sigs.kubespray.cluster
```

> Then run `ansible-playbook -i INVENTORY --become --become-user=root PLAYBOOK`
> — your inventory/group_vars stay in your repo.

Key Takeaways

- **Clone + pin a release tag**, then `pip install -r requirements.txt` inside a **Python venv** — never run off a moving `master`.
- **Copy** `inventory/sample` → `inventory/mycluster`; edit only your copy.
- Build `hosts.yaml` with the **inventory builder** or edit by hand; groups define topology and `k8s_cluster` **is the derived union**.
- Use **≥ 3 etcd**; `ansible_host` / `ip` / `access_ip` separate SSH, bind, reach.
- **Ping with** `all -m ping -b` before `cluster.yml`; `--become` is required.

End of Lesson 3

Next: Lesson 4 — Cluster Configuration

`group_vars/all/all.yml` · `group_vars/k8s_cluster/k8s-cluster.yml`